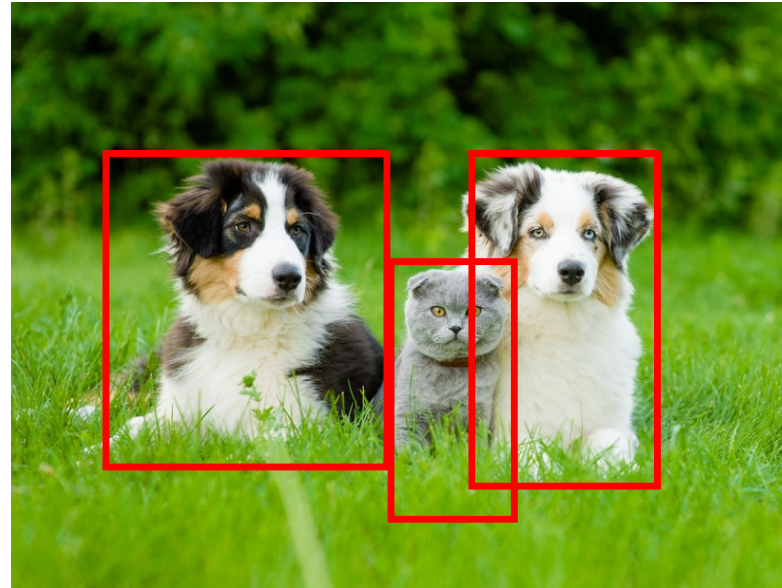


Finding and classifying objects



Information Technology

March 27, 2025

Problems in Computer Vision

Image Classification

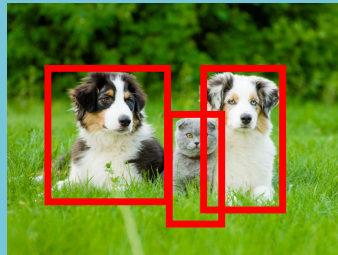
Image $\mapsto$ Class



last weeks, yesterday

Object Detection

Image $\mapsto$ List[(Region, Class)]



last week ago, today

Traditionally:
Features + machine learning
Cascaded classifier

Since ~2015:
2-step and 1-step
detectors (regions, CNNs)

Semantic Segmentation

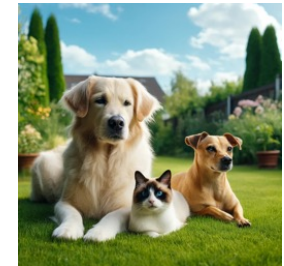
Image $\mapsto$ Array[Class]



14 Nov

Image Generation

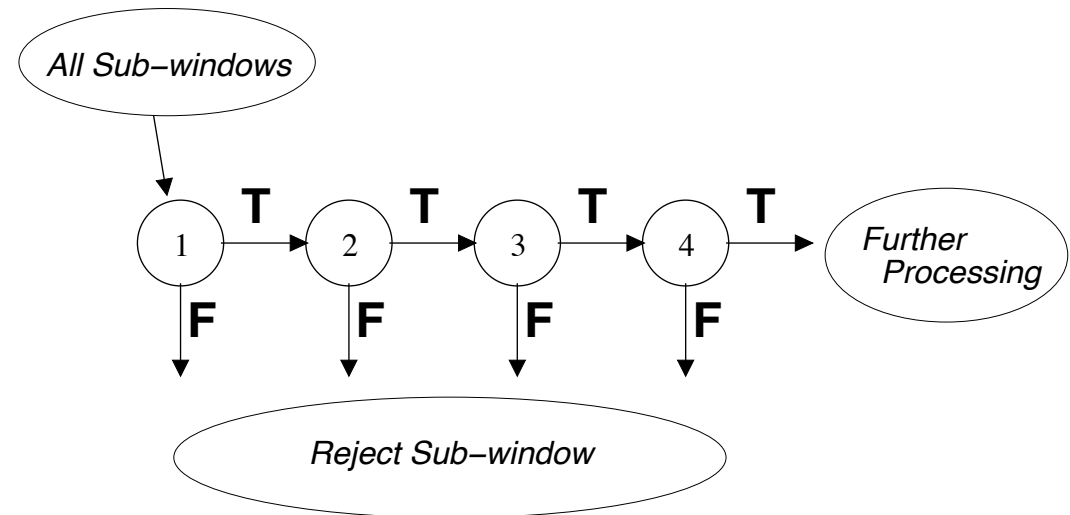
Text $\mapsto$ Image



15 Nov

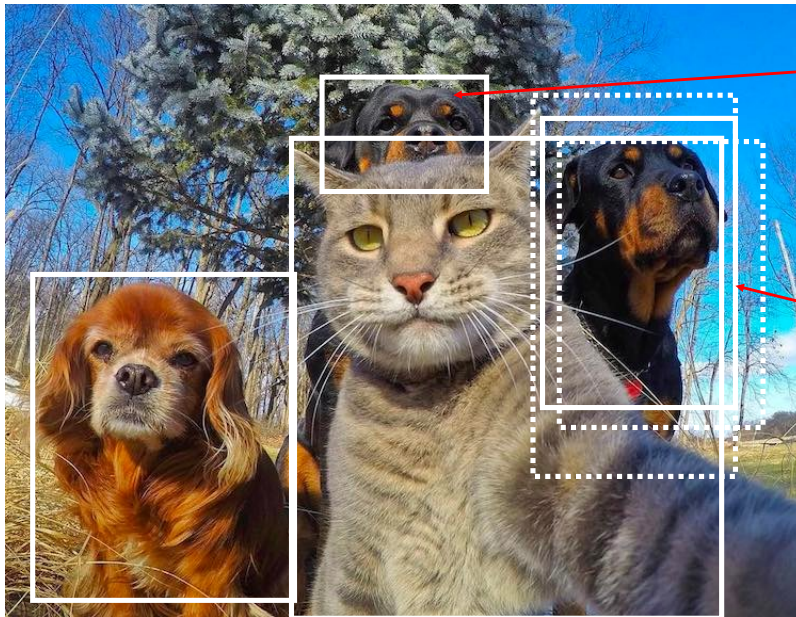
Classical Approach: Cascaded Classifier

- Sliding Window Approach
- Classification:
- Features (Haar Wavelets)
- Classifier (Boosting)
- Cascade for eliminating negative regions quickly



Classification and Localization

Finding multiple objects can be interpreted as a classification and localization task

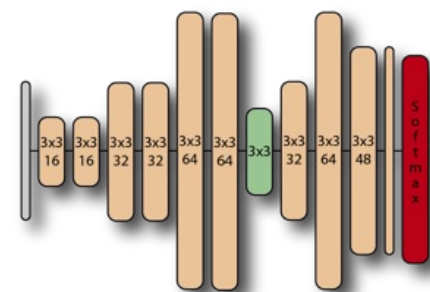


Correct class: **Classification**

Best Bounding Box: **Localization**

Detection as Classification: Sliding Windows

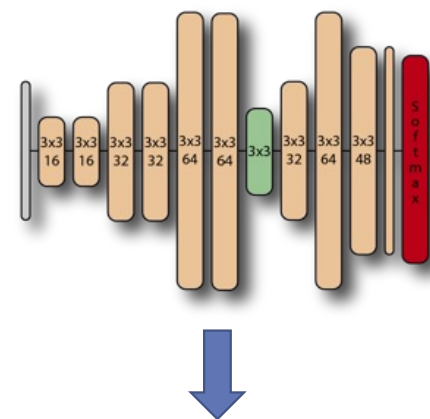
Sliding Window Approach: Apply CNN to many different sub-windows



Dog: **NO**
Cat: **NO**
Background: **YES**

Detection as Classification: Sliding Windows

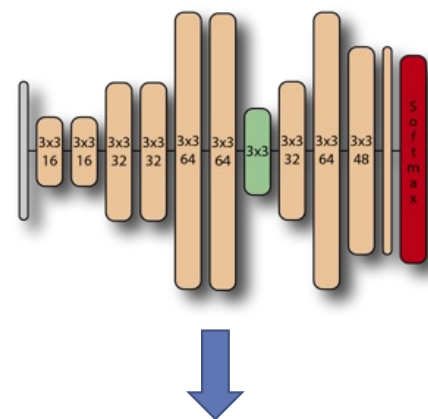
Sliding Window Approach: Apply CNN to many different sub-windows



Dog: **YES**
Cat: **NO**
Background: **NO**

Detection as Classification: Sliding Windows

Sliding Window Approach: Apply CNN to many different sub-windows



Dog: **NO**
Cat: **YES**
Background: **NO**

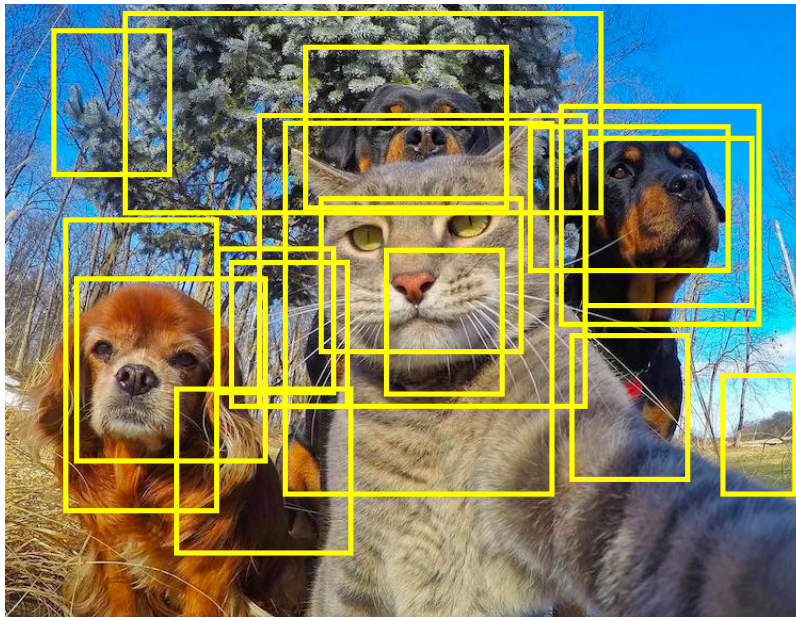
Detection as Classification: Sliding Windows

Problem:

Need to apply CNN to a huge number of different positions and sizes



Region Proposals



Region proposal approach:

- Find image regions that are likely to contain an object
- Relatively fast for 1000 proposals

Some approaches:

Alexe et al, "Measuring the objectness of image windows", TPAMI 2012

Uijlings et al, "Selective Search for Object Recognition", IJCV 2013

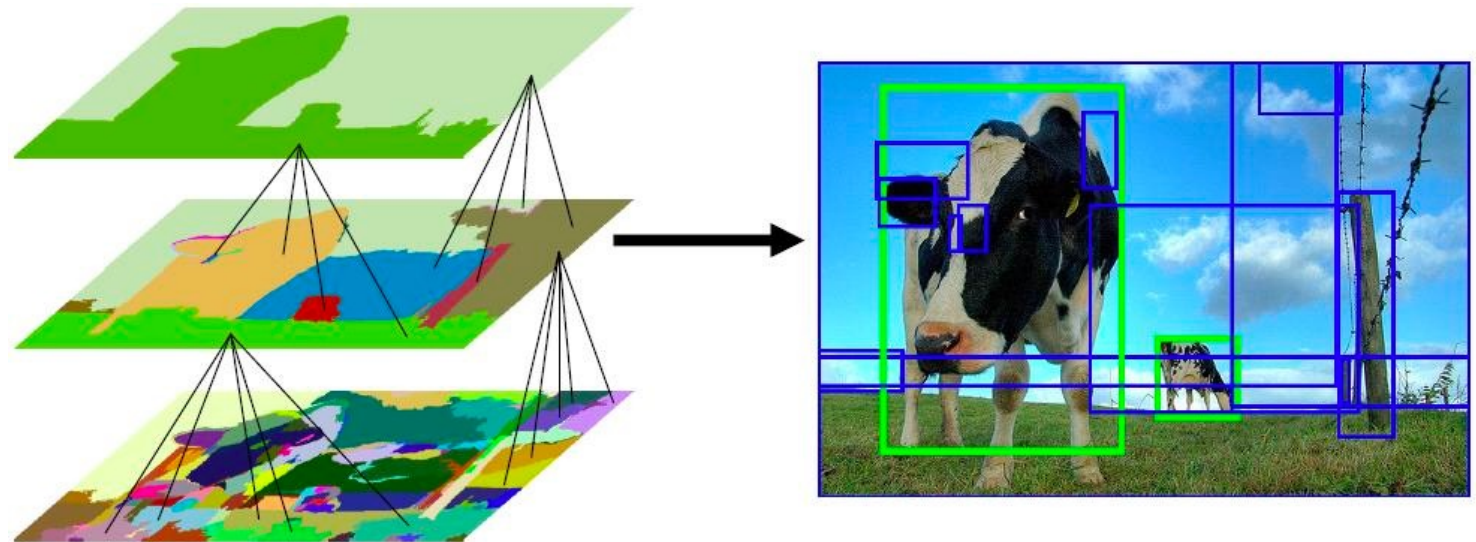
Cheng et al, "BING: Binarized normed gradients for objectness estimation at 300fps", CVPR 2014

Zitnick and Dollar, "Edge boxes: Locating object proposals from edges", ECCV 2014

Region Proposals by selective Search

Region Segmentation (Graph based)

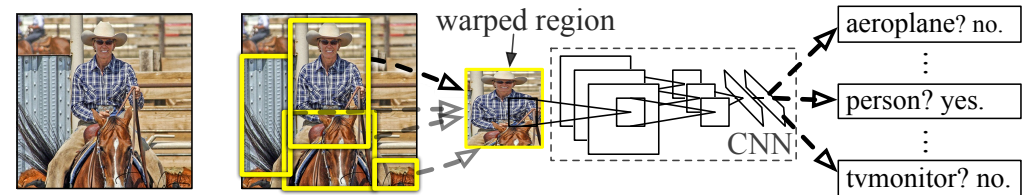
Hierarchical Grouping



R-CNN

Approach:

- Extract region proposals
- Compute CNN Features
- Classify Regions using SVM
- "Synergy of deep architectures with classical computer vision"



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.

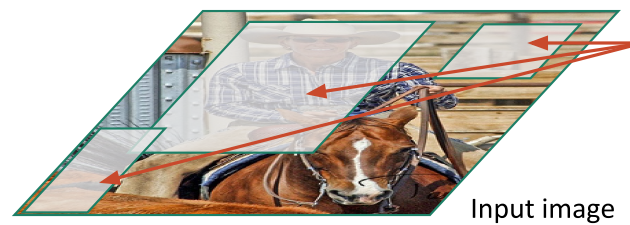
R-CNN



Input image

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

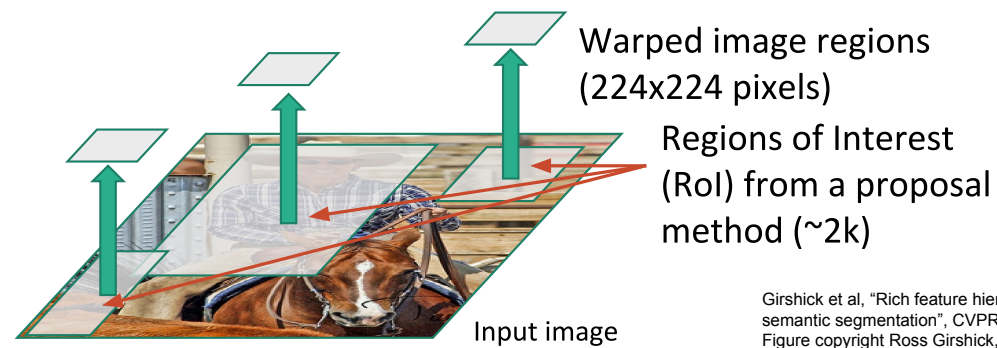
R-CNN



Regions of Interest
(RoI) from a proposal
method (~2k)

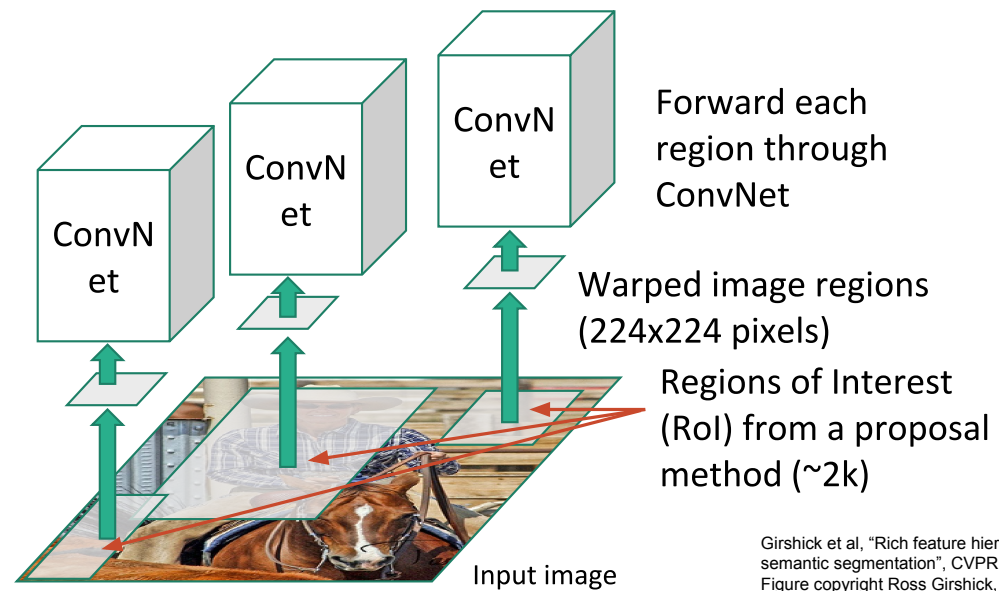
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN



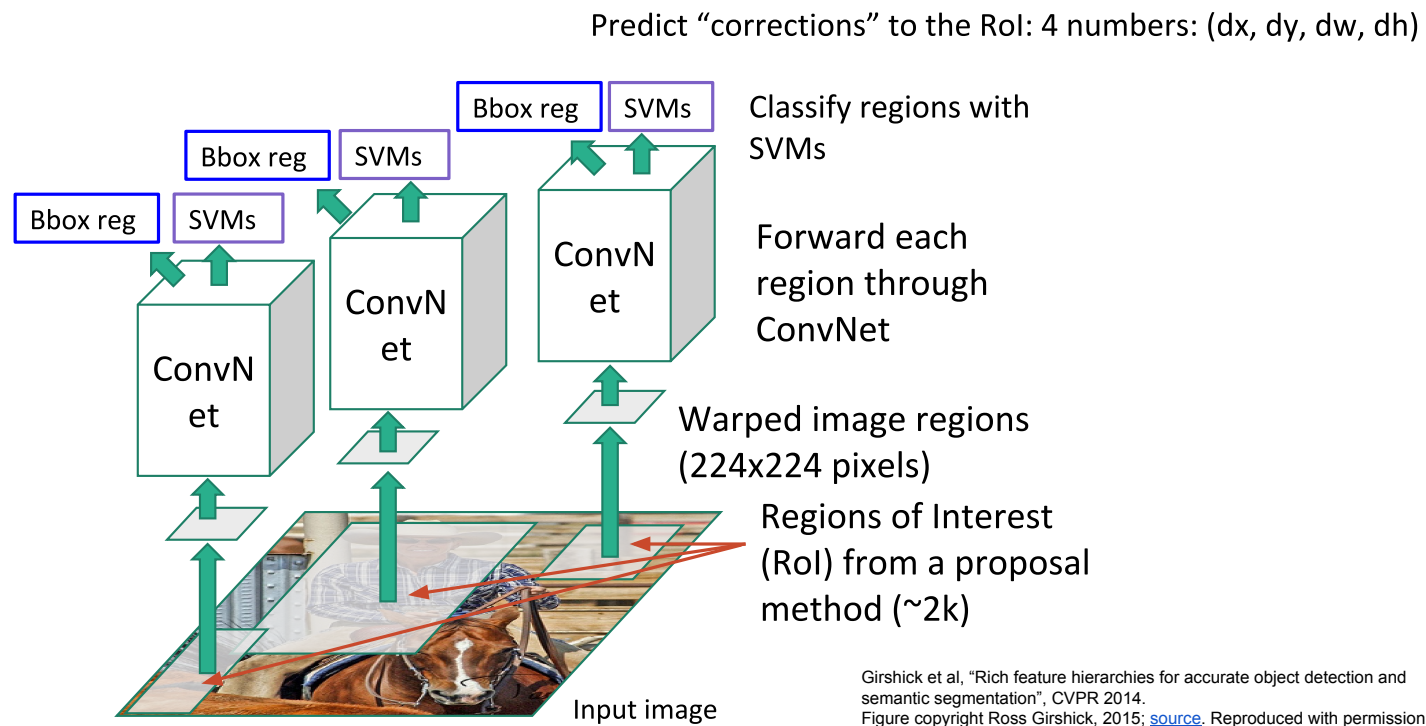
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN



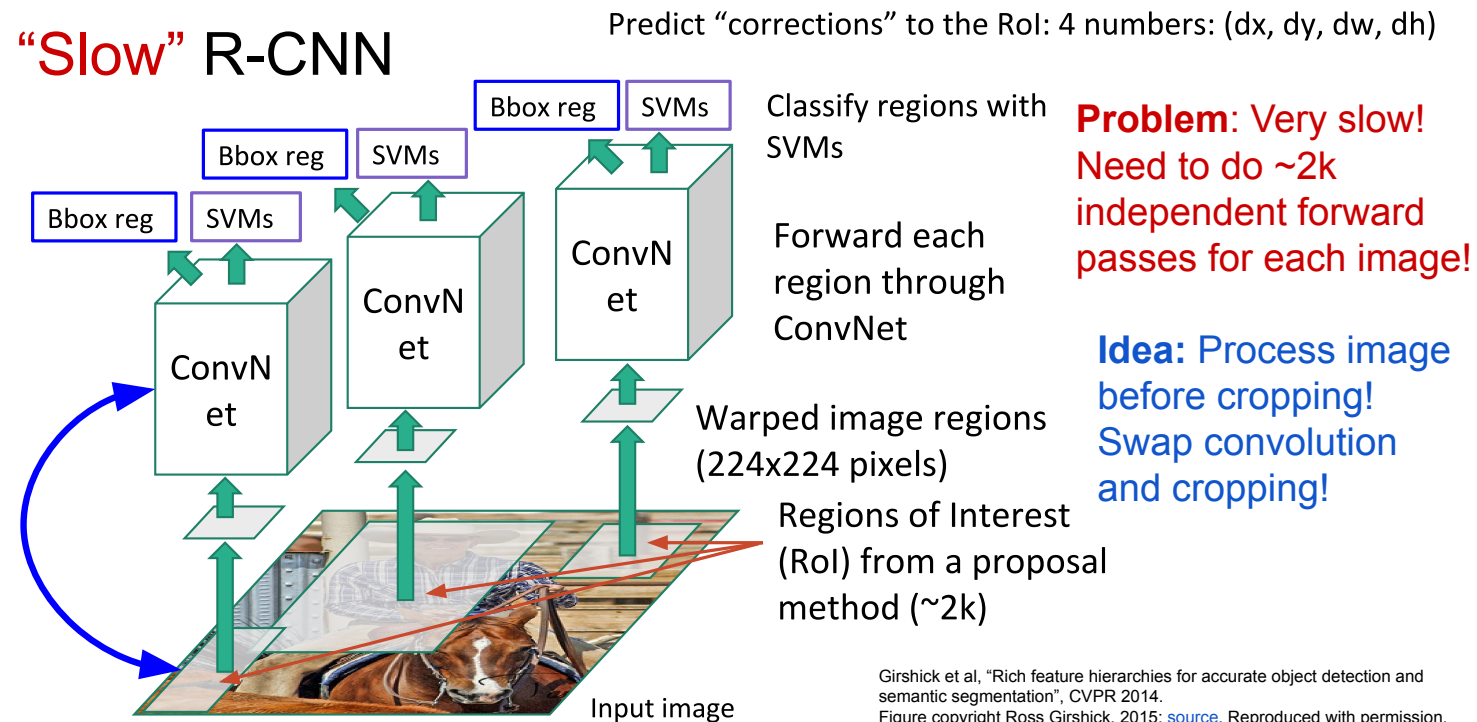
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN



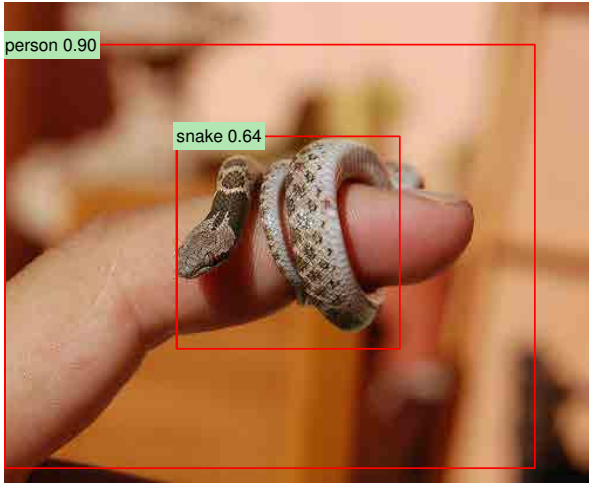
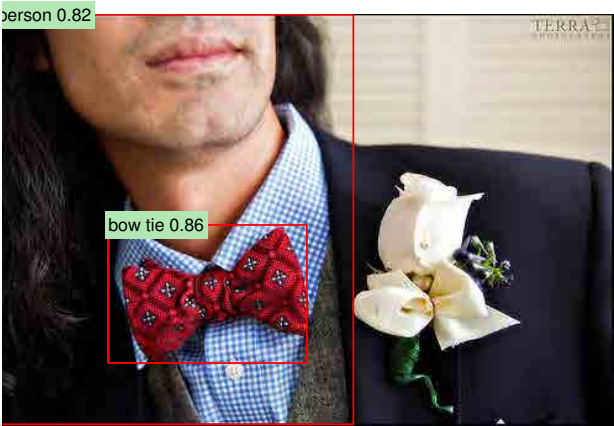
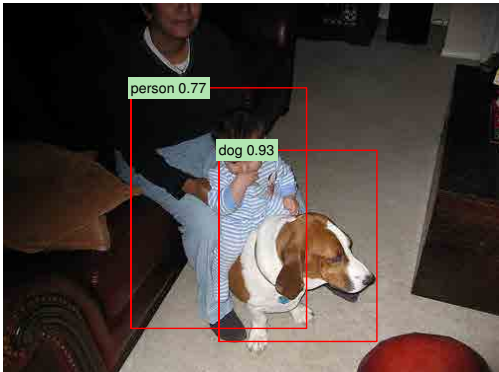
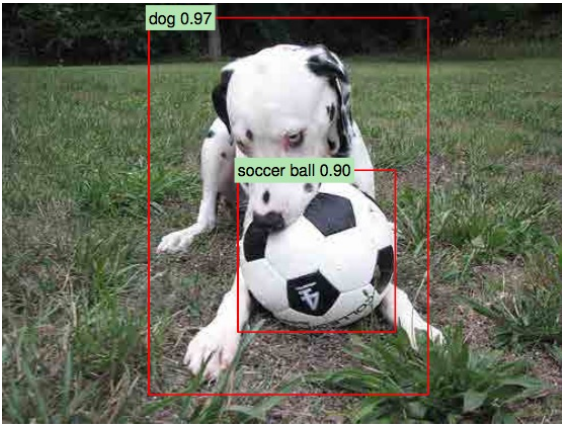
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN Results



R-CNN Details

Feature Extraction:

- Proposal regions are warped to 227x227 images
- 4096 Features from 227x227 RGB Image using 5 Convolutional and 2 Fully connected layers from AlexNet Architecture
- Supervised pre-training: on ILSVRC2012 classification data set
- Domain-specific fine-tuning: continue training on warped region proposals

Classifier:

- Linear SVM per class: 1 vs. all others

Classifier training:

- Regions with overlap of 30% is considered a positive example

Bounding box position:

- Treat as regression problem

Bounding Box Regression

- Given a (initial) predicted bounding box

$$\mathbf{p} = (p_x, p_y, p_w, p_h)$$

and a ground truth bounding box

$$\mathbf{g} = (g_x, g_y, g_w, g_h)$$

learn a transformation to align them.

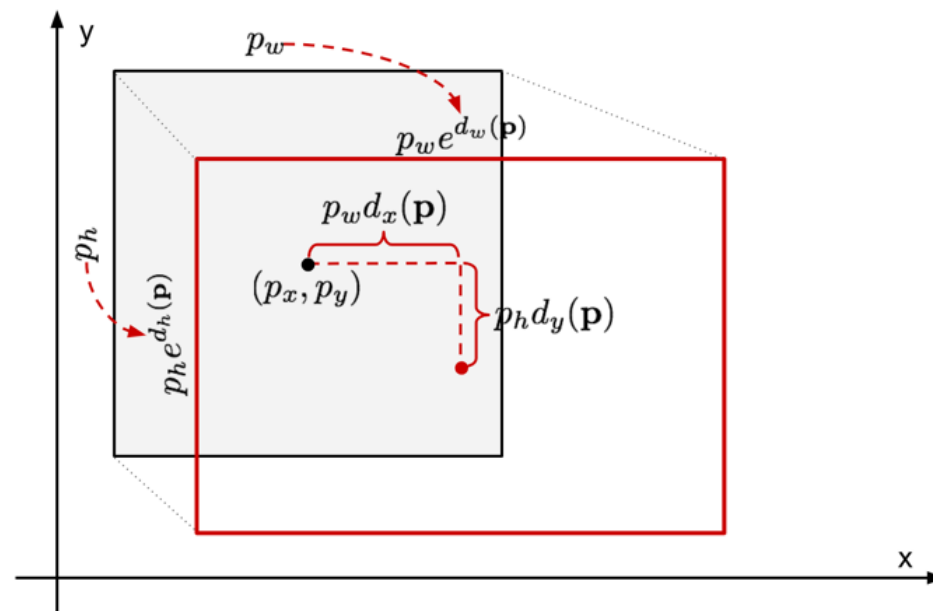
- The transformations d_* are formulated, so that they are scale invariant

$$\hat{g}_x = p_w d_x(\mathbf{p}) + p_x$$

$$\hat{g}_y = p_h d_y(\mathbf{p}) + p_y$$

$$\hat{g}_w = p_w \exp(d_w(\mathbf{p}))$$

$$\hat{g}_h = p_h \exp(d_h(\mathbf{p}))$$



Bounding Box Regression

The regression loss can be formulated as

$$\mathcal{L}_{\text{reg}} = \sum_{i \in \{x, y, w, h\}} (t_i - d_i(\mathbf{p}))^2 + \lambda \|\mathbf{w}\|^2$$

with the targets calculated from

$$t_x = (g_x - p_x) / p_w$$

$$t_y = (g_y - p_y) / p_h$$

$$t_w = \log(g_w / p_w)$$

$$t_h = \log(g_h / p_h)$$

i.e. the d_* are learned functions, i.e. the regression head of the network

CNN only for feature extraction?

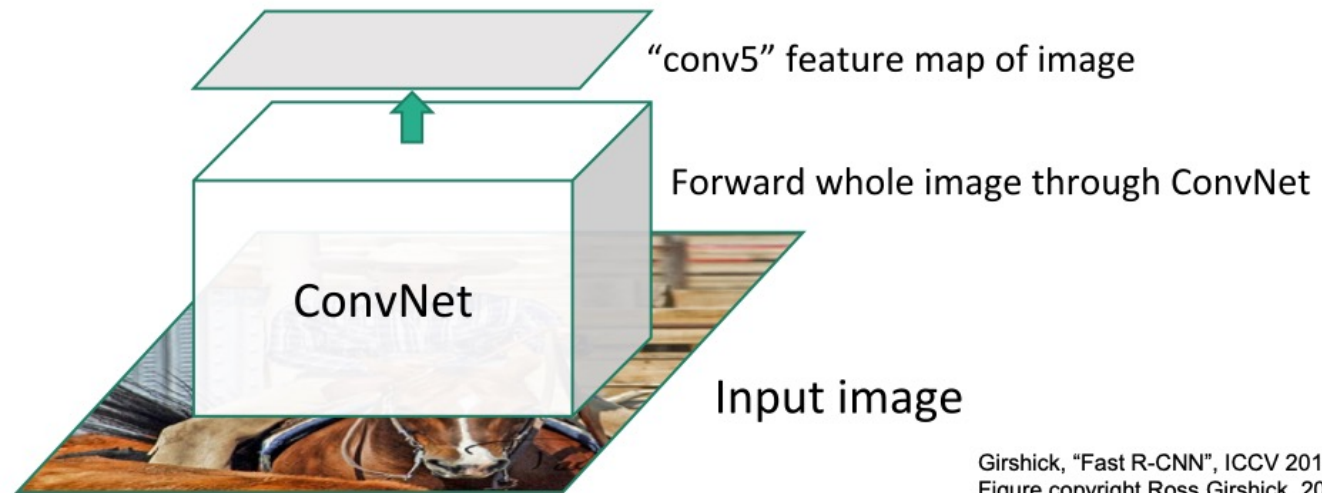


Fast R-CNN

Replace SVM or R-CNN with fully connected neural network for

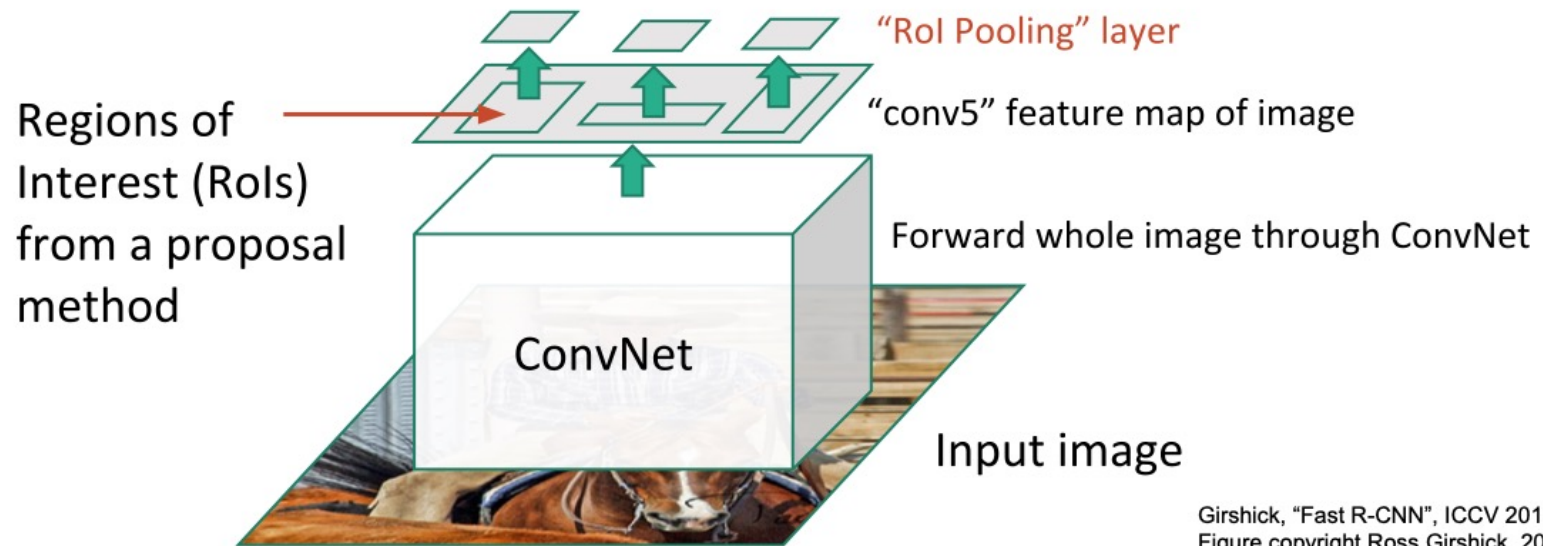
- Classification of object
- Refine bounding box of object

Fast-R-CNN



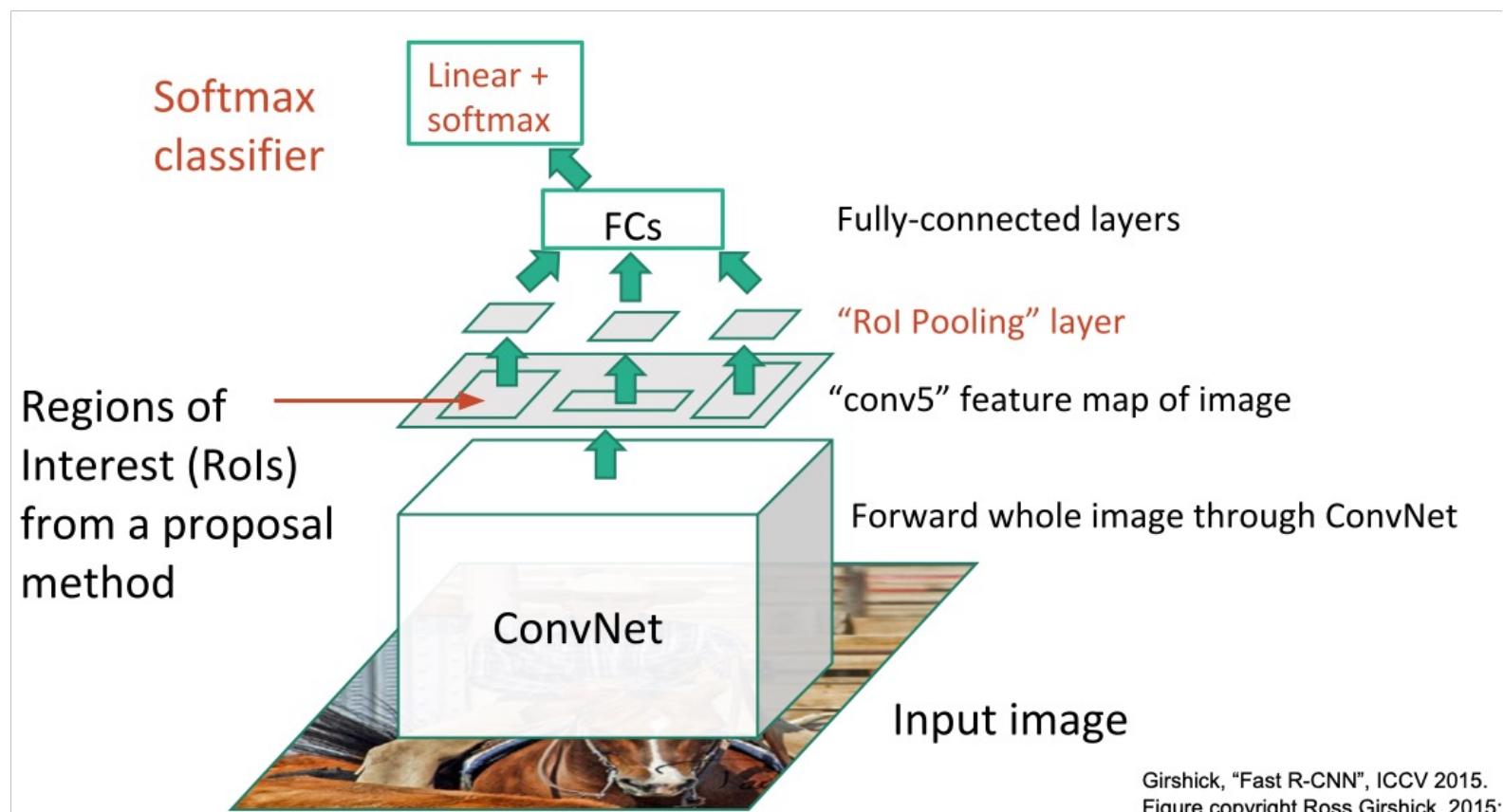
Girshick, "Fast R-CNN", ICCV 2015.
Figure copyright Ross Girshick, 2015;

Fast-R-CNN



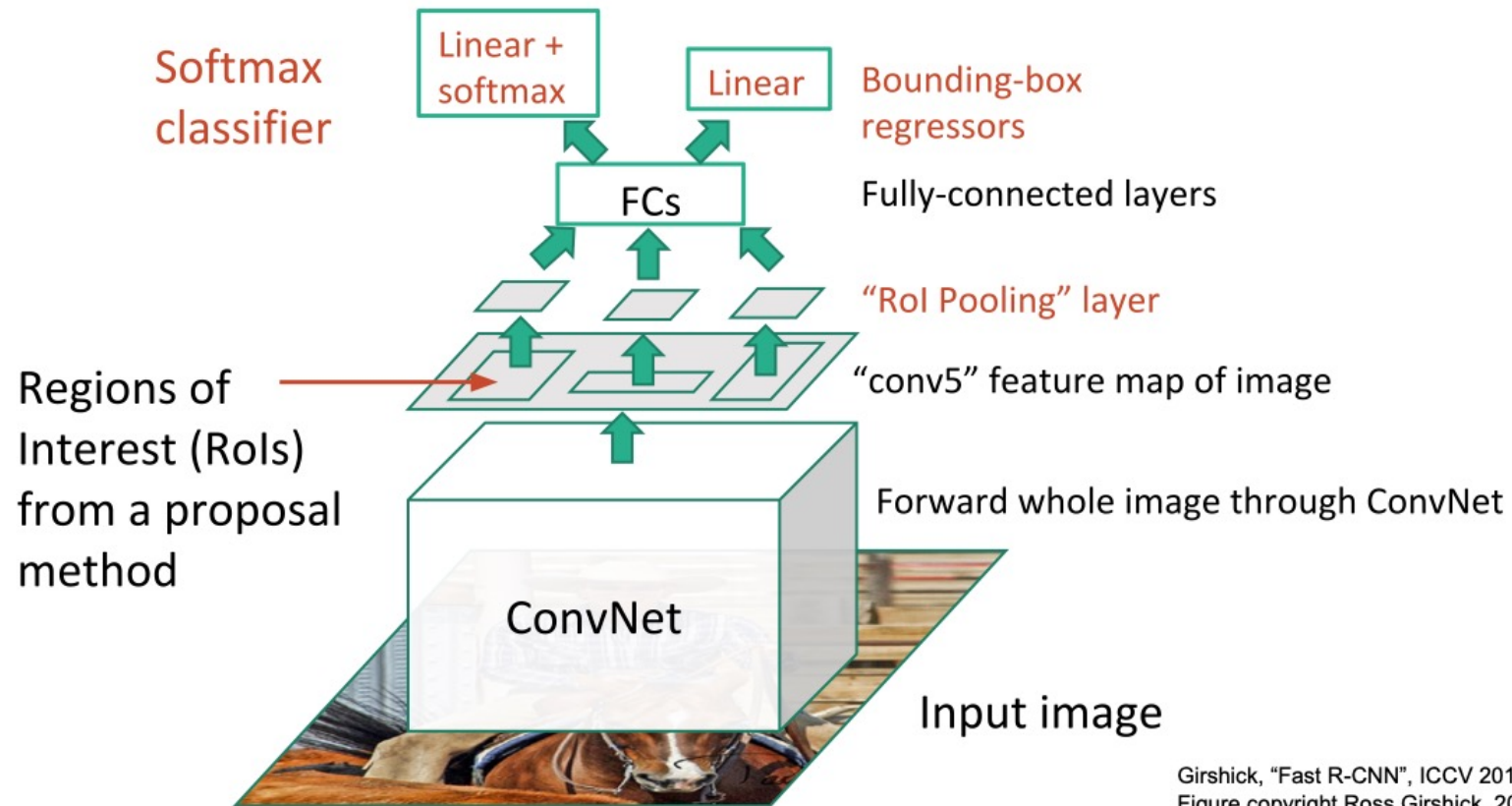
Girshick, "Fast R-CNN", ICCV 2015.
Figure copyright Ross Girshick, 2015;

Fast-R-CNN



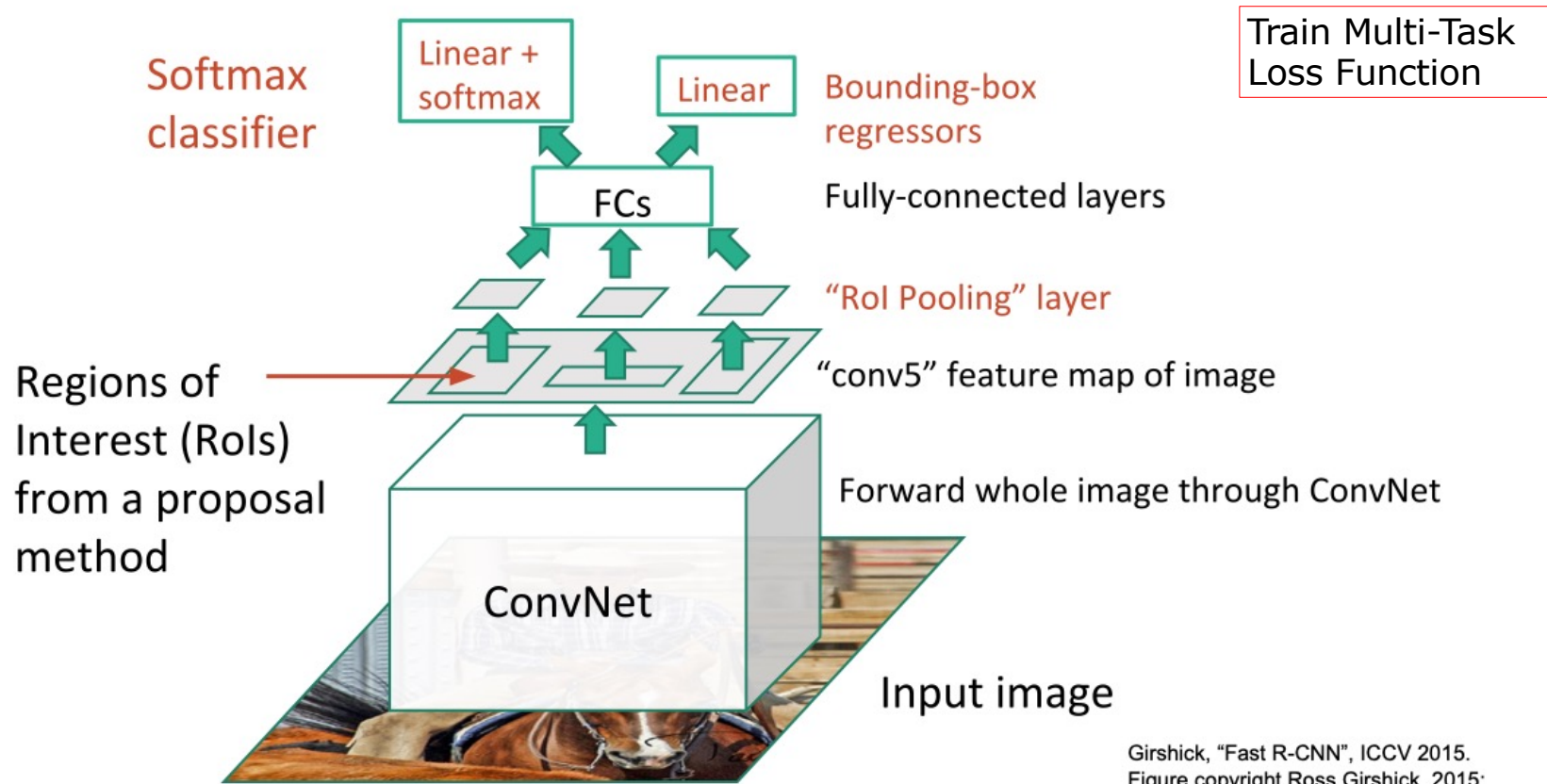
Girshick, "Fast R-CNN", ICCV 2015.
Figure copyright Ross Girshick, 2015;

Fast-R-CNN



Girshick, "Fast R-CNN", ICCV 2015.
Figure copyright Ross Girshick, 2015;

Fast-R-CNN



Girshick, "Fast R-CNN", ICCV 2015.
Figure copyright Ross Girshick, 2015;

Faster R-CNN

Faster R-CNN:
Make CNN do proposals!

Faster R-CNN is a
Two-stage object detector

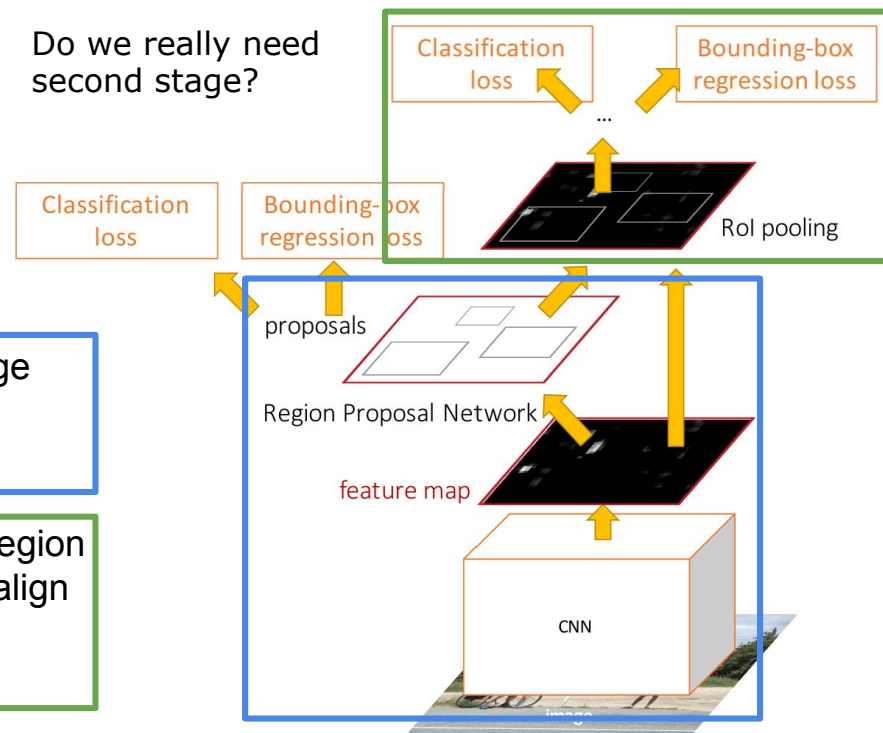
First stage: Run once per image

- Backbone network
- Region proposal network

Second stage: Run once per region

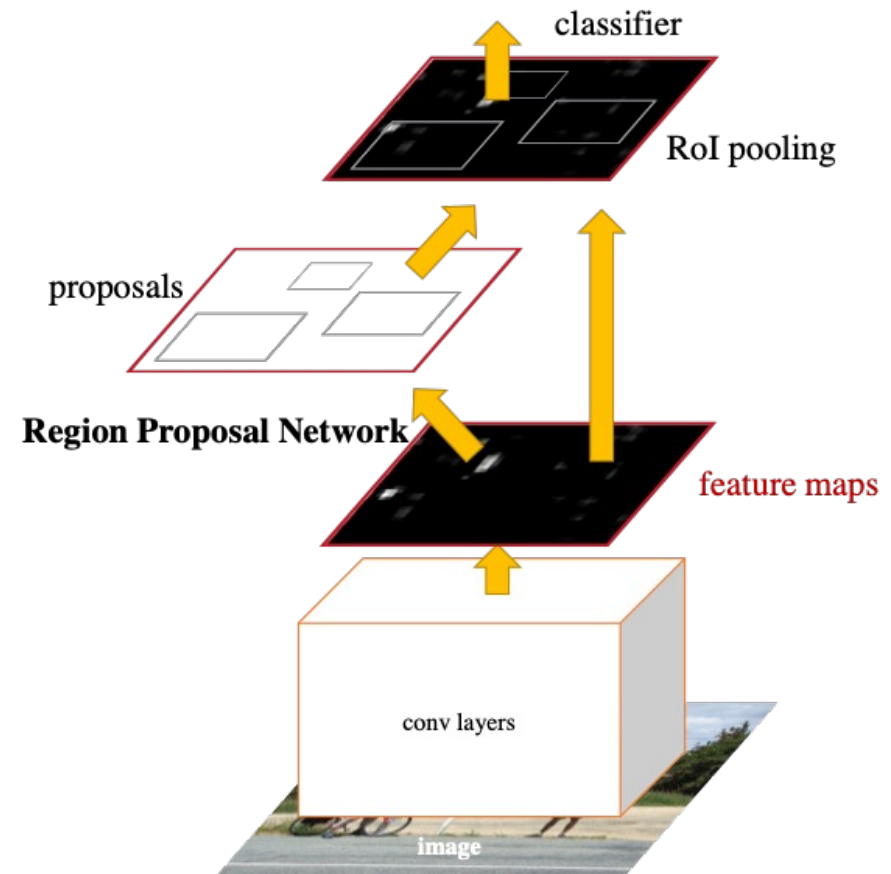
- Crop features: RoI pool / align
- Predict object class
- Prediction bbox offset

Do we really need
second stage?



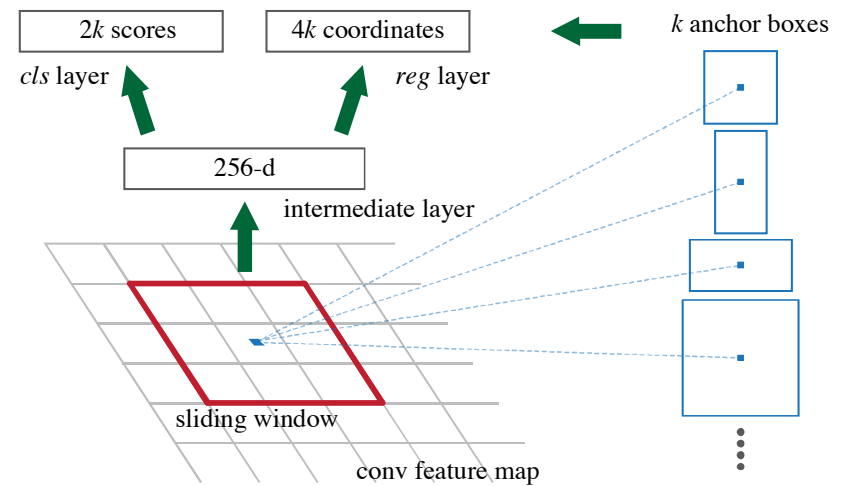
Faster R-CNN

- Replace proposal region function with fully convolutional network → Region Proposal Network (RPN)
- Then applies the Fast R-CNN detector using the proposed regions.



Faster R-CNN: Region Proposals

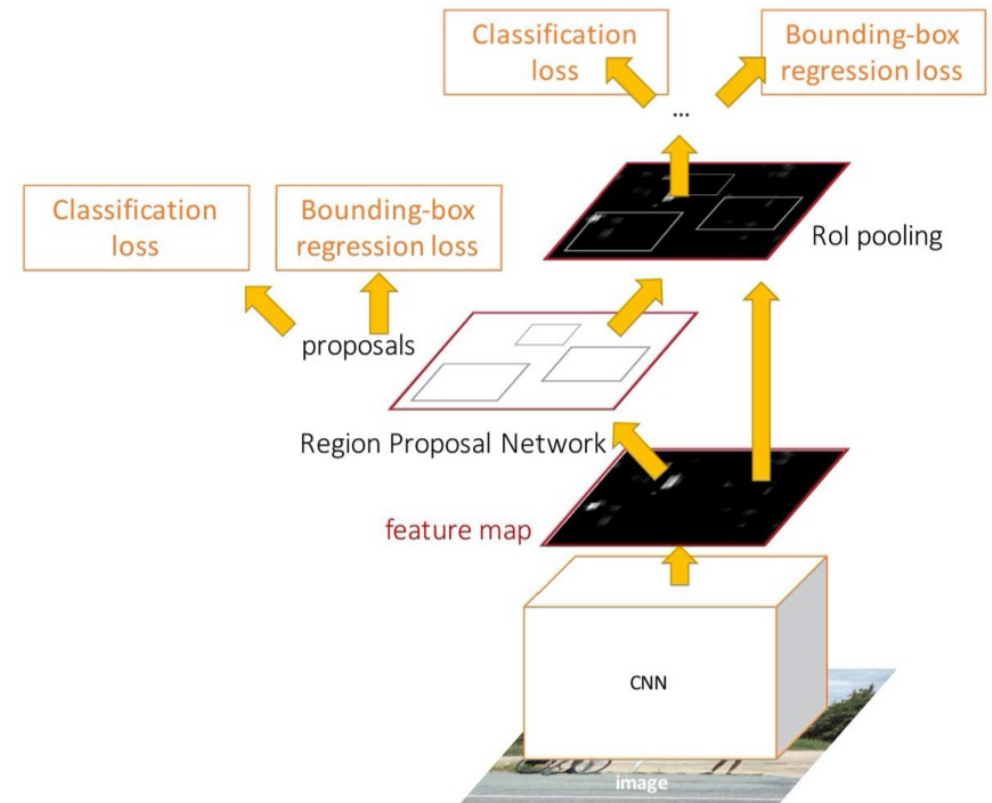
- The proposal network uses the idea of a *sliding window* with fully connected box-class and box regression layers for each window
- (This can be implemented with a convolutional layer followed by 1×1 convolutions)
- Each position predicts k bounding boxes and the probability of object/no object
- The k bounding boxes are predicted as relative transformations of k anchor boxes at different scales and aspect ratio



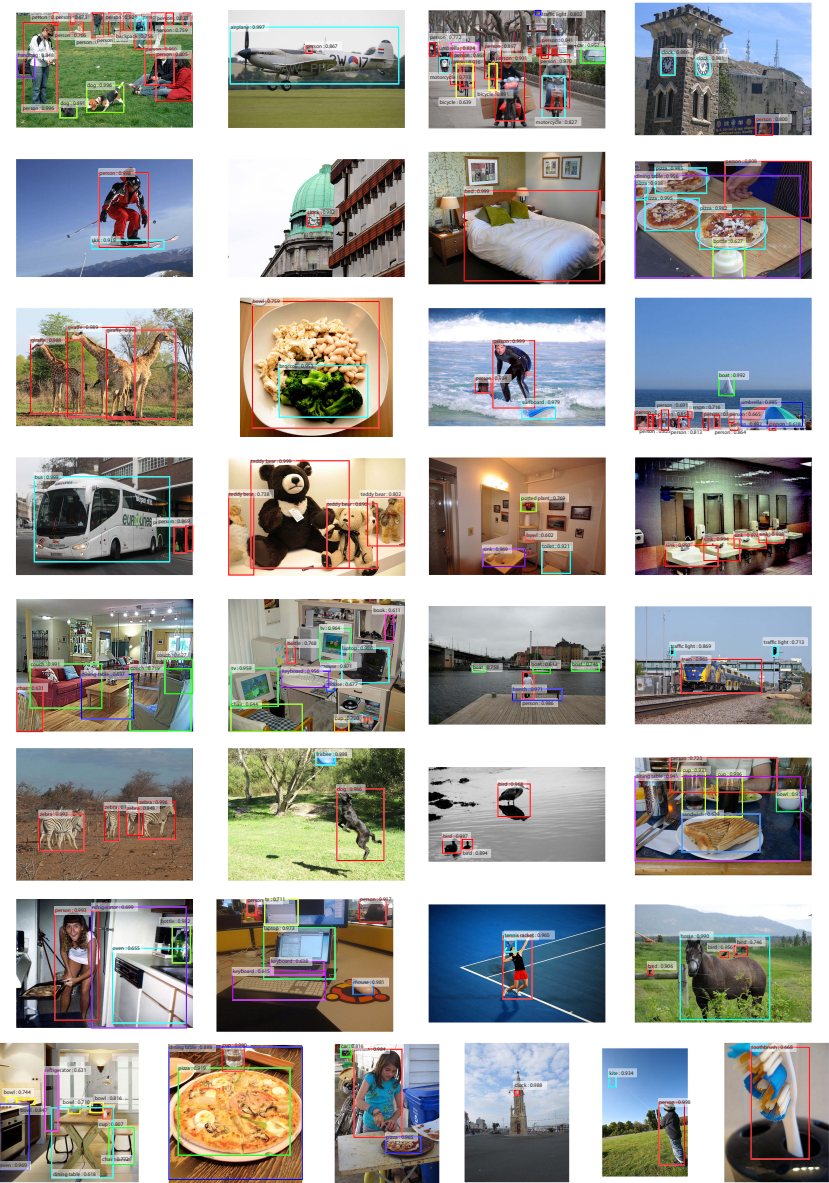
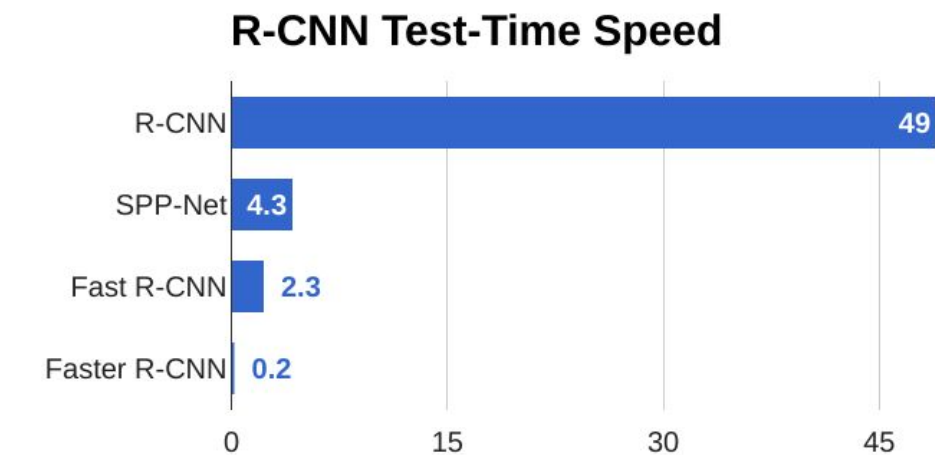
Faster C-RNN

Train 4 losses:

- RPN classification object / no object
 - RPN regress box coordinates
 - Final classification (object classes)
 - Final bounding box
-
- Uses alternating training of the RPN and the detector network first
 - Then combines the network to have shared features



Speed Comparison



Single-State Detectors: YOLO (You Only Look Once)

Approach:

- Divide Image into $S \times S$ grid
- Each grid cell predicts B bounding boxes with confidence scores (x, z, w, h, score)
- Each grid cell predicts class probabilities (independent of bounding boxes)
- Multiply confidence score and probabilities at test time

YOLO

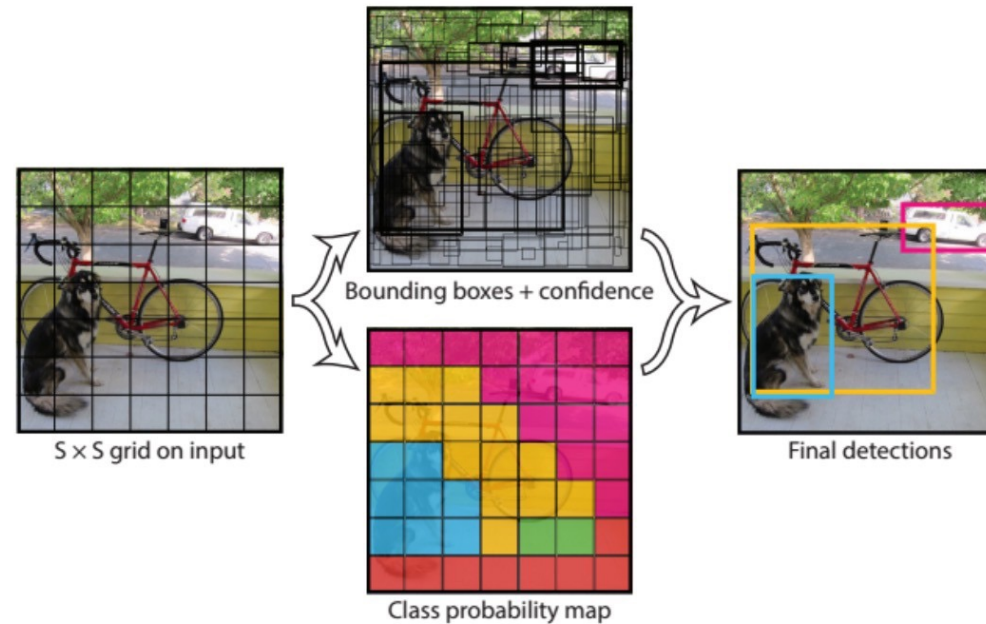
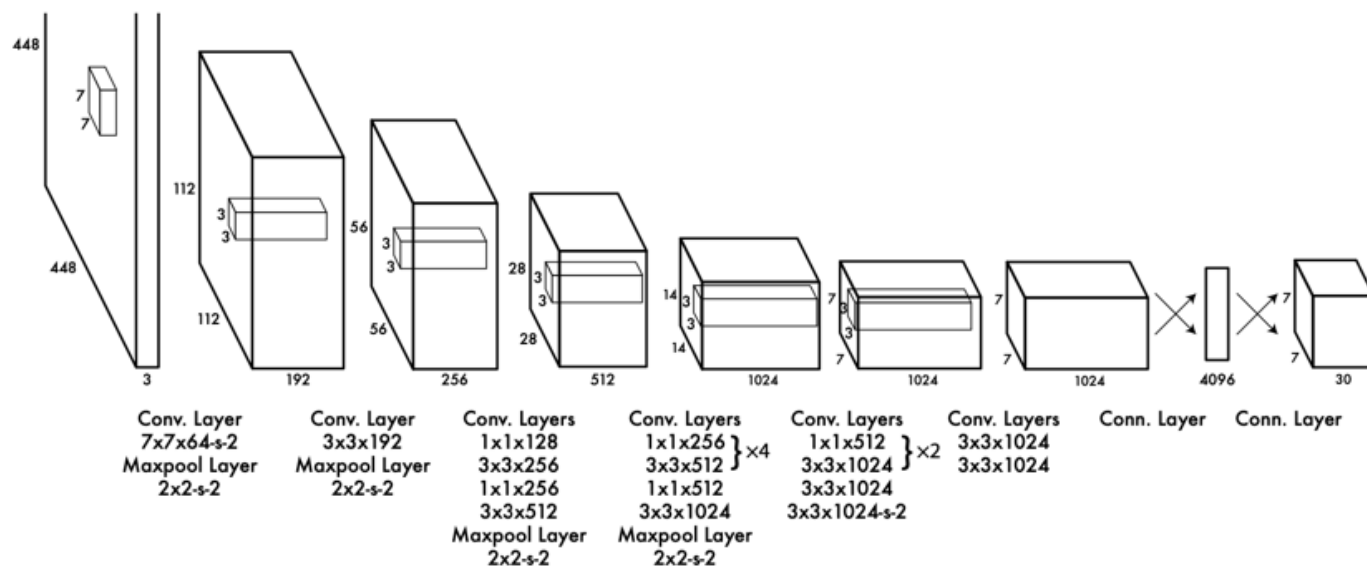


Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

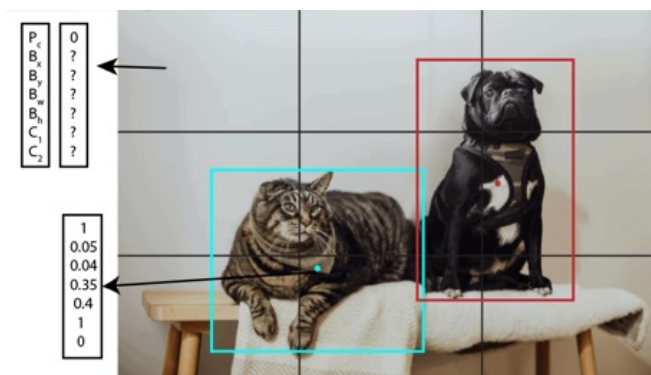
How does YOLO Work: Backbone



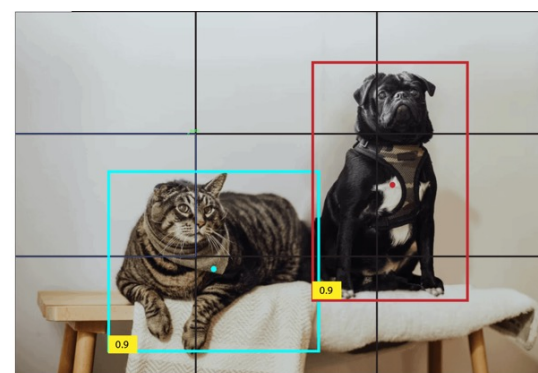
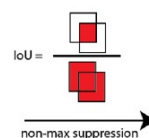
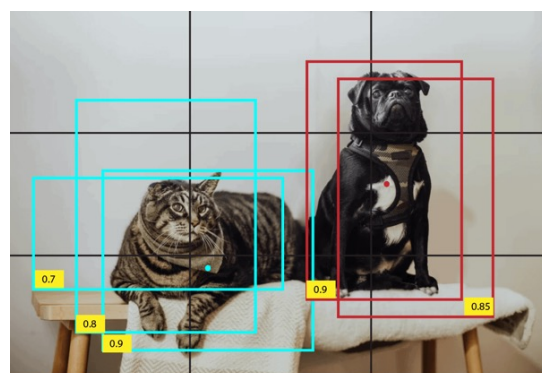
- Encode Bounding Boxes and Class Probabilities for each cell
- Loss Function compares the output with ground truth

Yolo V1 Backbone

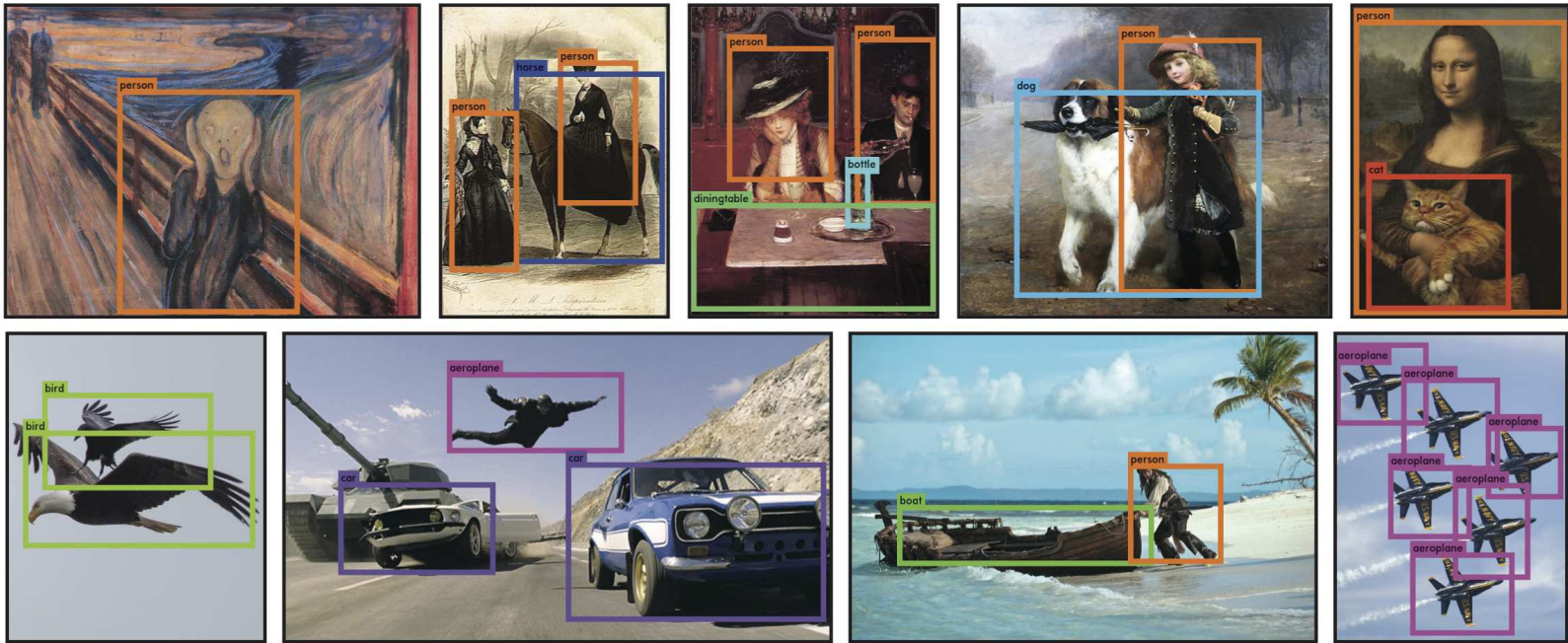
How does YOLO Work: Non-Max Suppression



- Non-Max Suppression cleans overlapping bounding boxes
- Only the best result remains



YOLO Results



Yolo Variants

V2:

- Darknet 19 Backbone, Batch Normalization
- Anchor Boxes

V3:

- Darknet 58 Backbone, residual connections

V4

- CSP Darknet Backbone, spatial attention
- Mish activation,

V5:

- EfficientNet Backbone
- GIoU Loss function

V6:

- EfficientNet Light Backbone

V7:

- CSPDarknet-Z Backbone
- Leaky relu activation function

V8:

- Multiscale Object Detection, ELU Activation Function, General IoU Loss function

V11:

- More efficient architecture

Yolo Base nets: Darknet 19 and 58

Type	Filters	Size/Stride	Output
Convolutional	32	3×3	224×224
Maxpool		$2 \times 2/2$	112×112
Convolutional	64	3×3	112×112
Maxpool		$2 \times 2/2$	56×56
Convolutional	128	3×3	56×56
Convolutional	64	1×1	56×56
Convolutional	128	3×3	56×56
Maxpool		$2 \times 2/2$	28×28
Convolutional	256	3×3	28×28
Convolutional	128	1×1	28×28
Convolutional	256	3×3	28×28
Maxpool		$2 \times 2/2$	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Maxpool		$2 \times 2/2$	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	1000	1×1	7×7
Avgpool		Global	1000
Softmax			

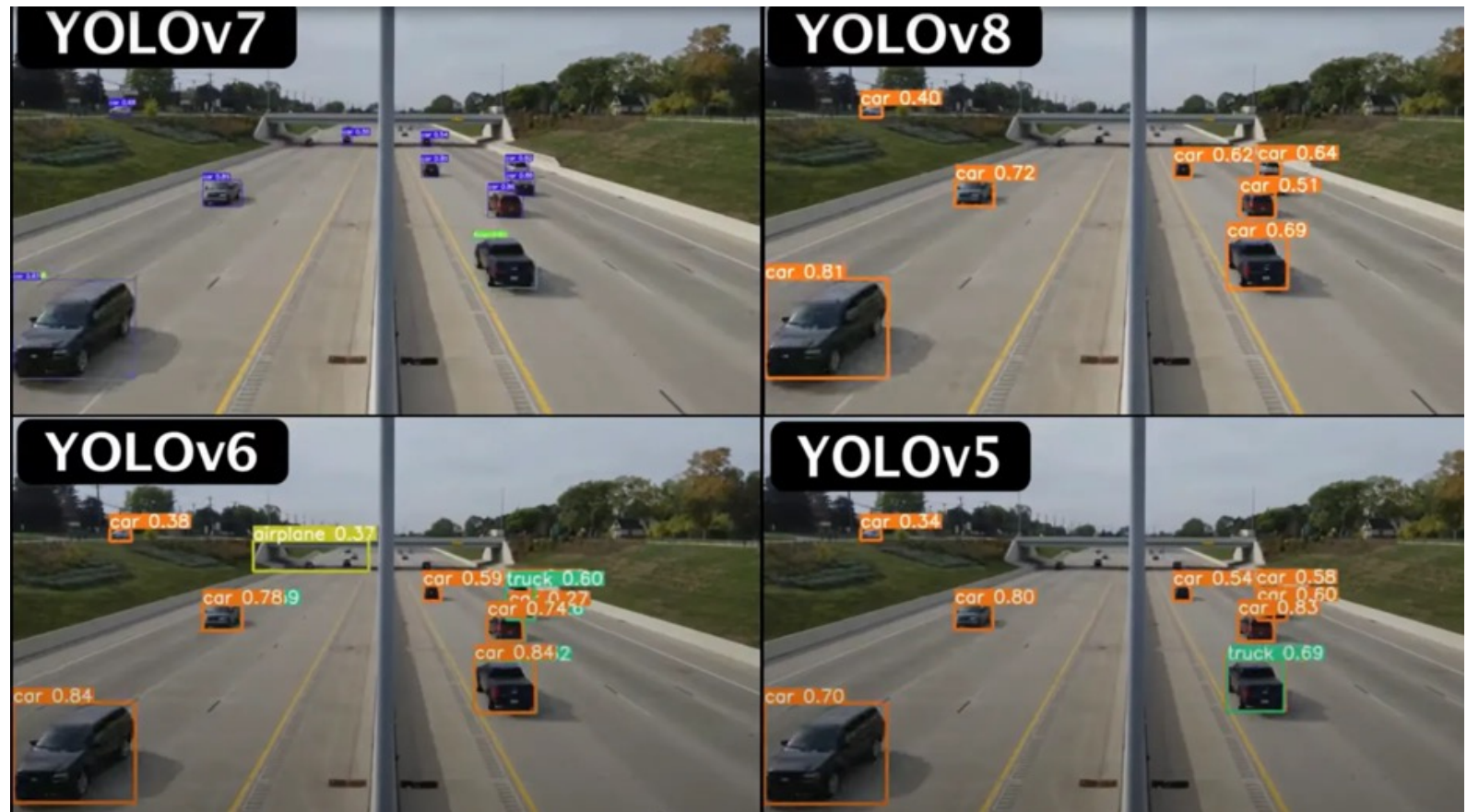
Type	Filters	Size	Output
Convolutional	32	3×3	256×256
Convolutional	64	$3 \times 3 / 2$	128×128
Convolutional	32	1×1	
1x Convolutional	64	3×3	
Residual			128×128
Convolutional	128	$3 \times 3 / 2$	64×64
Convolutional	64	1×1	
2x Convolutional	128	3×3	
Residual			64×64
Convolutional	256	$3 \times 3 / 2$	32×32
Convolutional	128	1×1	
8x Convolutional	256	3×3	
Residual			32×32
Convolutional	512	$3 \times 3 / 2$	16×16
Convolutional	256	1×1	
8x Convolutional	512	3×3	
Residual			16×16
Convolutional	1024	$3 \times 3 / 2$	8×8
Convolutional	512	1×1	
4x Convolutional	1024	3×3	
Residual			8×8
Avgpool		Global	
Connected		1000	
Softmax			

Yolo V8

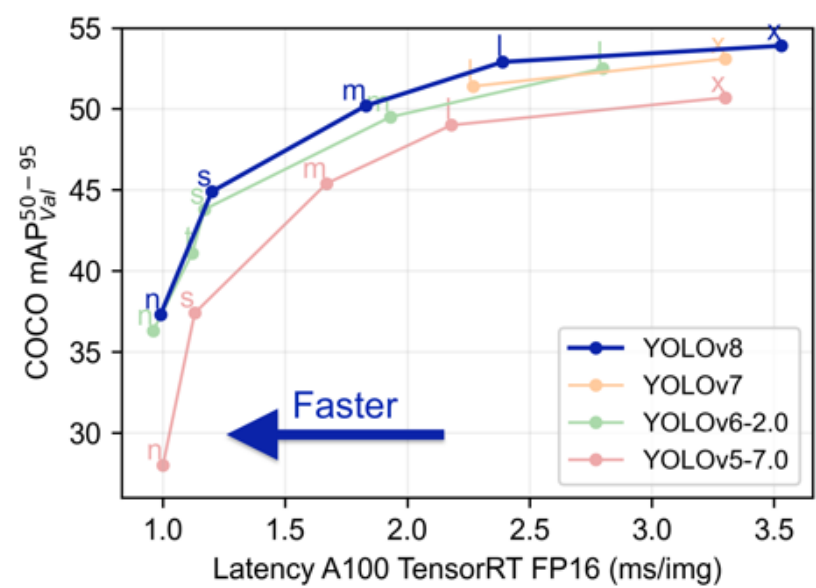
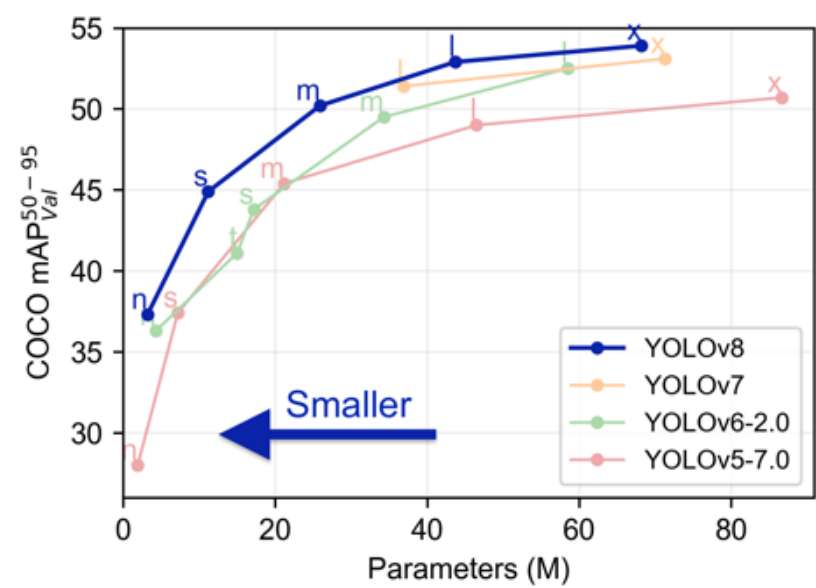
Multiscale Object Detection

ELU Activation Function

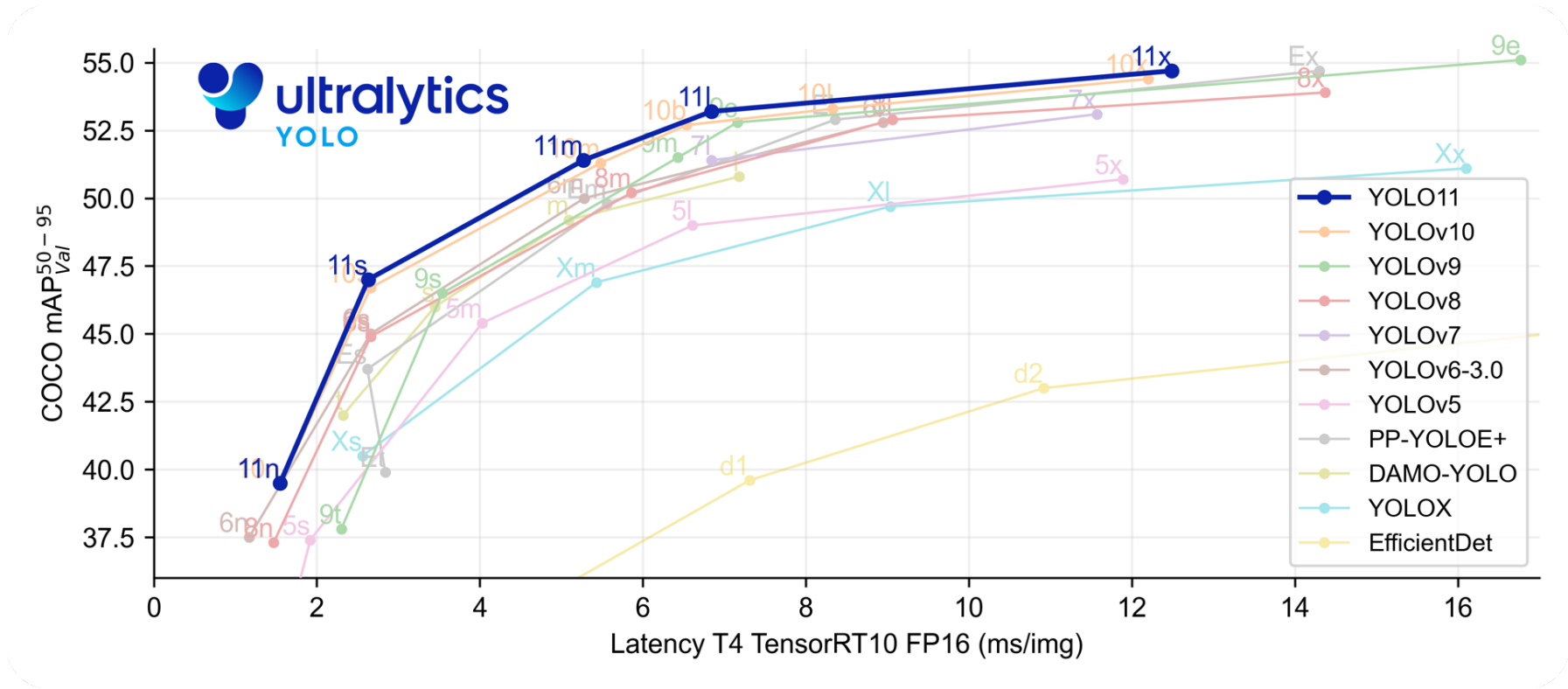
General IoU Loss function



YOLO v8 Results



Yolo V11 Results (newest version ☺)

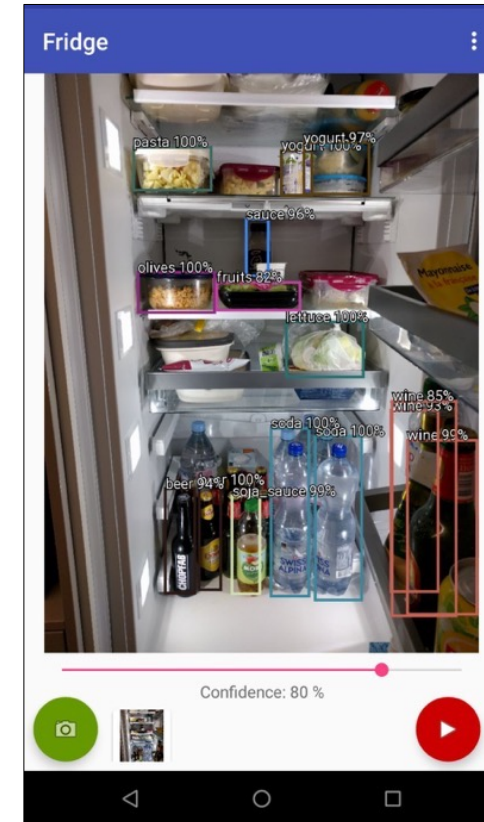


Deep Neural Fridge Analyzer



Bachelor Thesis HSLU:

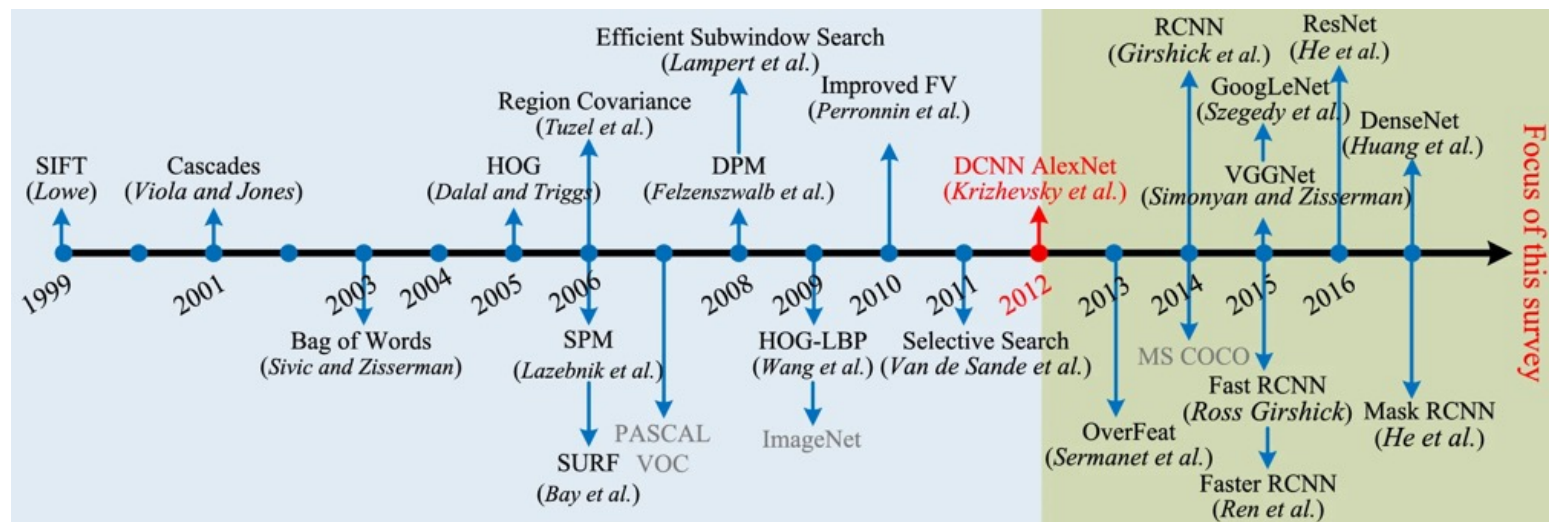
- Marc Bravin
- Marvin Walker



Comparison

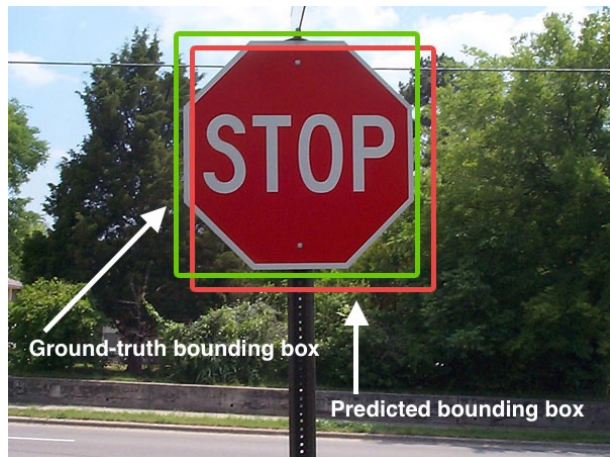
	Validation Set		
Ground Truth	SSD	R-FCN	YOLO
			

Overview of Object Detection Approaches



Assessment of Object Detection

Object Detection is usually measured as mAP @ IoU 0.5

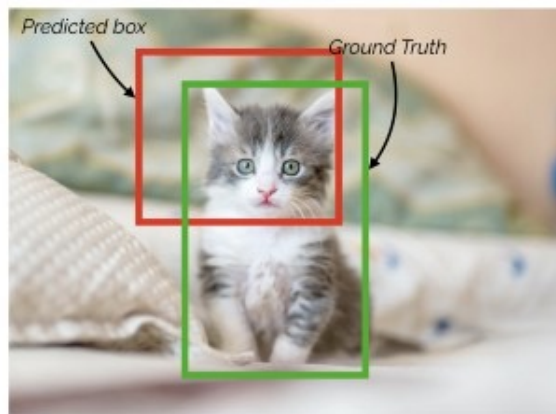


$$\text{Intersection over Union (IoU)} = \frac{\text{Area of Overlap}}{\text{Area of Union}} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

IoU Thresholds

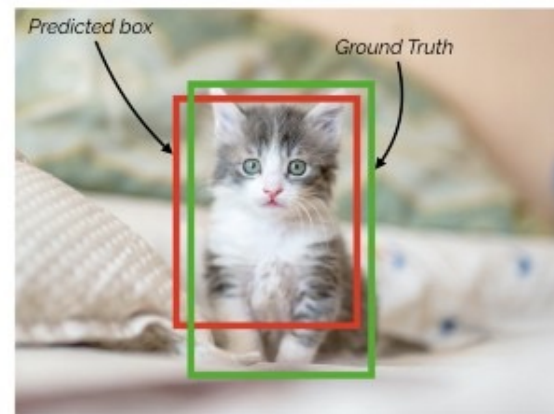
If IoU threshold = 0.5

False Positive (FP)



$IoU = \sim 0.3$

True Positive (TP)



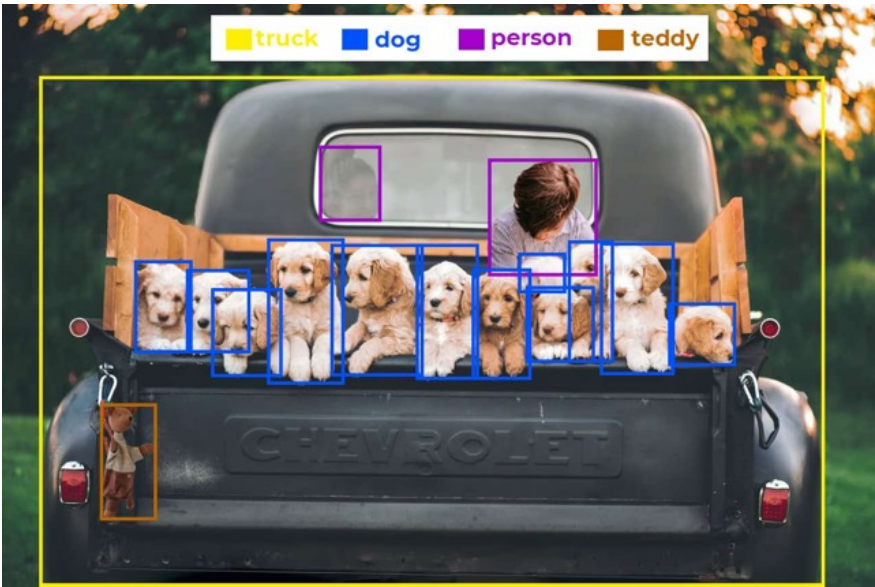
$IoU = \sim 0.7$

Calculating Mean Average Precision (mAP)

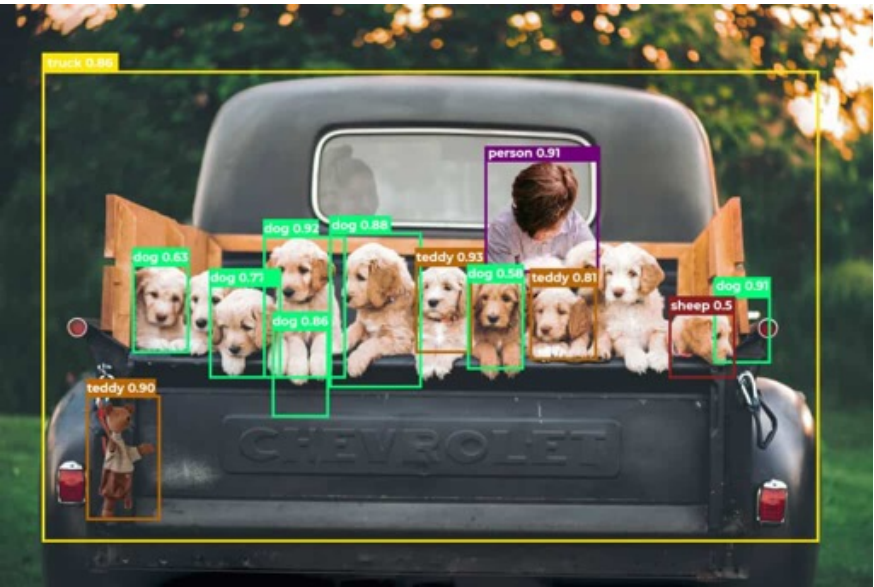
- Mean average precision is the mean of the average precision (AP) of each class
- So calculate AP first for each class
- Then take the mean over all classes

- Usually computed at different IoU thresholds

Average Precision



Ground Truth



Prediction

Average Precision

1. Sort Detection by score

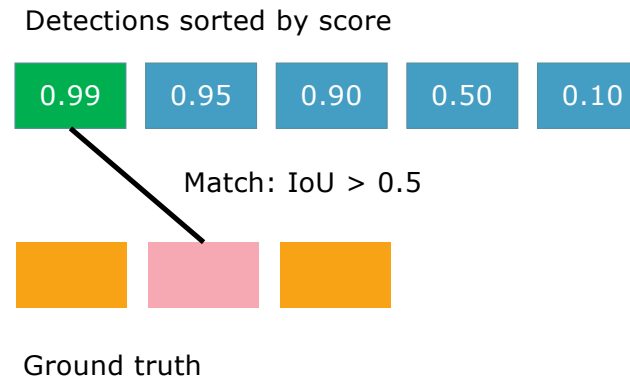
Detections sorted by score



Ground truth

Average Precision

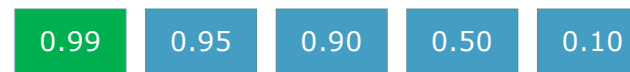
1. Sort Detection by score
2. For each detection (higher scores first)
 1. If there is a match with a ground truth box with $\text{IoU} > 0.5$, mark it as positive
 2. Otherwise mark it as negative



Average Precision

1. Sort Detection by score
2. For each detection (higher scores first)
 1. If there is a match with a ground truth box with $\text{IoU} > 0.5$, mark it as positive
 2. Otherwise mark it as negative
 3. Plot a point on Precision/Recall Curve

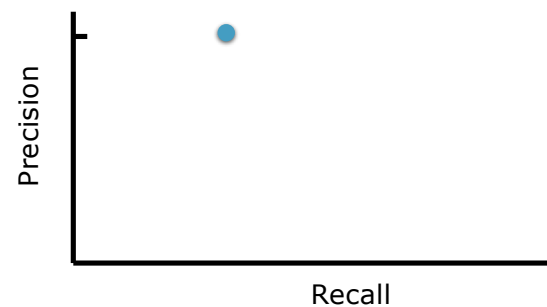
Detections sorted by score



Match: $\text{IoU} > 0.5$



Ground truth

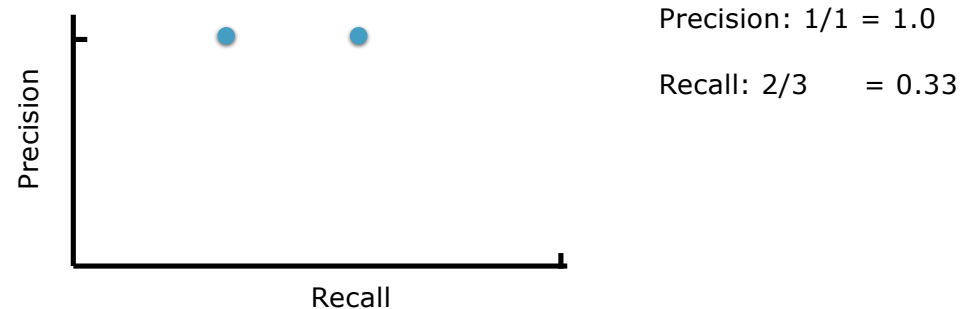
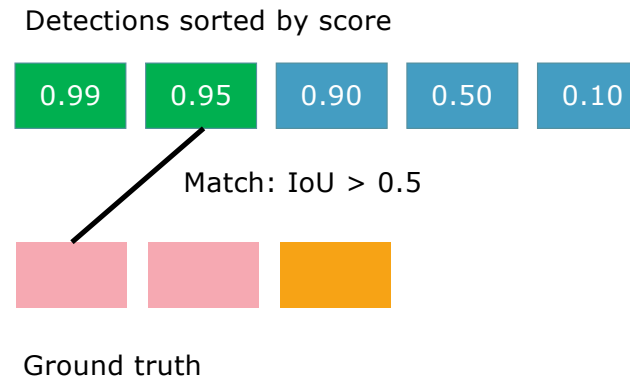


Precision: $1/1 = 1.0$

Recall: $1/3 = 0.33$

Average Precision

1. Sort Detection by score
2. For each detection (higher scores first)
 1. If there is a match with a ground truth box with $\text{IoU} > 0.5$, mark it as positive
 2. Otherwise mark it as negative
 3. Plot a point on Precision/Recall Curve



Average Precision

1. Sort Detection by score
2. For each detection (higher scores first)
 1. If there is a match with a ground truth box with $\text{IoU} > 0.5$, mark it as positive
 2. Otherwise mark it as negative
 3. Plot a point on Precision/Recall Curve

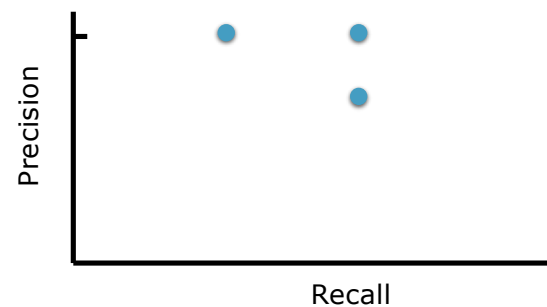
Detections sorted by score



No Match with $\text{IoU} > 0.5$



Ground truth



Precision: $2/3 = 0.67$

Recall: $2/3 = 0.67$

Average Precision

1. Sort Detection by score
2. For each detection (higher scores first)
 1. If there is a match with a ground truth box with $\text{IoU} > 0.5$, mark it as positive
 2. Otherwise mark it as negative
3. Plot a point on Precision/Recall Curve

Detections sorted by score



No Match with $\text{IoU} > 0.5$



Ground truth



Precision: $2/4 = 0.5$

Recall: $2/3 = 0.67$

Average Precision

1. Sort Detection by score
2. For each detection (higher scores first)
 1. If there is a match with a ground truth box with $\text{IoU} > 0.5$, mark it as positive
 2. Otherwise mark it as negative
 3. Plot a point on Precision/Recall Curve

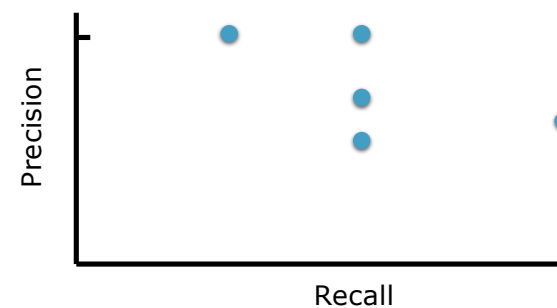
Detections sorted by score



Match: $\text{IoU} > 0.5$



Ground truth



Precision: $3/5 = 0.6$

Recall: $3/3 = 1.0$

Average Precision

1. Sort Detection by score
2. For each detection (higher scores first)
 1. If there is a match with a ground truth box with $\text{IoU} > 0.5$, mark it as positive
 2. Otherwise mark it as negative
 3. Plot a point on Precision/Recall Curve
3. Average Precision (AP) = Area under PR curve

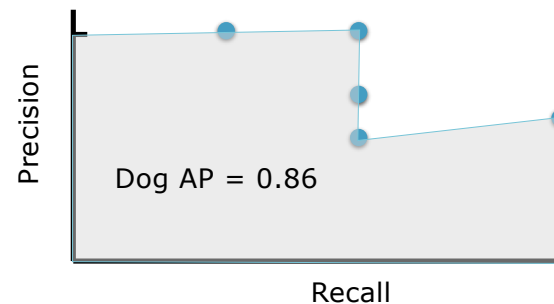
Detections sorted by score



Match: $\text{IoU} > 0.5$



Ground truth



Precision: $3/5 = 0.6$

Recall: $3/3 = 1.0$

Yolo Demo

No exercises, but you can try out Yolo on the server:

```
pip install ultralytics
```

```
yolo predict model=/exchange/cvai/models/yolov8n.pt source=/exchange/cvai/images/dogs_on_truck.jpg
```

See more on

<https://docs.ultralytics.com/modes/predict/>